Refining Competency-Based Grading in Undergraduate Programming Courses

Marisha Rawlins
Electrical and Computer Engineering
School of Engineering
Wentworth Institute of Technology
Boston, USA

Pilin Junsangsri
Electrical and Computer Engineering
School of Engineering
Wentworth Institute of Technology
Boston, USA

1 Introduction

Traditional grading and assessment in undergraduate university courses, including programming courses, usually use the following methodology. First, the instructor defines a grade breakdown containing their key course assessment types, for example, lab exercises are worth 30% of the overall course grade, the project 20%, assignments 20%, and quizzes and exams the remaining 30%. Then the individual assessments are graded on a point-score, for example, Assignment 1 is worth 40 pts and the midsemester exam is worth 100pts. Finally, an overall letter grade and associated GPA are assigned based on the course assessment weightings and points. This traditional method persists because instructors are used to this method in non-programming courses where this method makes sense, and in many universities, the syllabus template is broken up to facilitate the traditional methodology. While this methodology works well for most courses, programming is a skills-based or competency-based course where the topics are different skills that a student is expected to master. Skill-based courses, like programming and math courses, could be graded with a scheme where each skill presented is rated and used to form an overall course grade rather than an overall grade based on the assessment types. Additionally, the ACM/IEEE Computing Curricula Report in December 2020 [1] suggested a transition from knowledge-based learning to competency-based learning in Computing undergraduate programs including Computer Engineering. Recognizing that skill-based courses like programming need to be evaluated differently, and with this shift to redefining courses and curricula in a competency-based model, the grading, assessment, and our evaluation of these courses need to be reevaluated as well.

A natural fit is competency-based grading (CBG) [2][3][4] where students focus on mastering the individual topics, therefore, students must show their level of competence in the various topics or sub-topics by the end of the semester. For example, a topic User-defined Functions is graded on a scale of 0 No Submission, 1 Beginner, 2 Competent, and 3 Proficient in one of the courses used in this study. One advantage of competency-based grading is clearly defined learning outcomes and expectations where rubrics are developed and are made available to students. This paper investigates different methodologies for implementing competency-based grading (CBG) in programming courses by combining CBG with other assessment strategies.

We start with the initial implementation of competency-based grading in a Junior-level programming course in the Electrical Engineering and Computer Engineering programs in 2020 and 2021. A characteristic of CBG is flexibility; to accomplish flexibility these courses combined a flipped classroom and unlimited resubmissions so that students had the freedom to move through the entire course at their own pace and so that students can learn from their

feedback. One disadvantage of this approach was the burden of grading on the instructor, with a large number of submissions to grade, it can be difficult to give students feedback quickly. In this study, we tested the following refinements to the CBG approach (1) infinite resubmissions with a live-coding lab demonstration, and (2) only one resubmission per assessment. We compared the performance - overall grades and numbers of students receiving grades D, F, or Withdraw - for the initial implementation and our two refinements. In the case of allowing only one resubmission per assessment we observed no overall change in performance. This showed that moving from unlimited to one resubmission does not affect performance while still allowing some flexibility and the opportunity for students to master the topic after receiving feedback. In the case of adding a live-coding lab demonstration, instructors were able to quickly grade the lab exercise and give feedback directly to the students in real-time, decreasing the time taken to grade the lab exercises. Students also perform better when they have a live-coding demo or exam since they know that they will have to answer questions on their code in real-time. In future work, we plan to combine live-coding demonstrations with limited resubmissions.

2 Related Work

Our work combines competency-based grading with other techniques in an effort to find the right combined methodology for undergraduate programming courses at Wentworth Institute of Technology. This section briefly discusses the techniques used. For each, many examples of previous work exist, but in this paper, we only summarize the parts necessary to define our approach and do not do a full survey or analysis.

Competency-based grading (CBG)

In competency-based grading (CBG) [2][3][4][6], the course is broken up into the skills that a student should master by the end of the course, therefore, CBG fits well with skills-based courses like programming. In CBG every topic the student should master is graded based on a scale. Some work uses a binary scale, like satisfactory/unsatisfactory [4], and others use a larger scale such as 1 to 4 [3]. CBG typically uses clearly defined rubrics that are available to students while they are working on the assessment; because of this, students know the expectations. Since one goal of CBG is that students can master the different topics, CBG schemes usually allow students to resubmit so that they can learn from their mistakes and truly have a chance to master the topic. This also makes the course flexible since one mistake in an assessment does not necessarily have to impact the student's overall grade if they *choose* to resubmit and they achieve a higher grade. In CGB, the overall score, or student's letter grade, is a calculation based on the number of topics the student attempted and their mastery level for each topic.

Students' letter grades are still calculated when using CBG methodologies, this is because overall course grades usually factor into a student's GPA, therefore, instructors must submit a grade. However, the more important record for CBG is the list of individual competency levels achieved for the different topics. This helps the students keep track of their progress throughout the semester, and it lets the student know their strengths and weaknesses within the course after the semester. In programs that use credentialing where students are given an official record of their competencies per topic, students can use their mastery level when applying for jobs, and employers can use the credentials to look at the applicant's level of mastery in topics most applicable to the position.

Feedback-centered assessment

Feedback is an important tool for student learning. While feedback is not exclusive to CBG, it helps students achieve mastery [2][5][6]. The feedback from the instructor is used by the students to prepare for another resubmission in CBG.

Multiple resubmissions and Flexible deadlines

The number of resubmissions used in a course ranges from one [7] to infinite resubmissions [6]. Ideally, if flexibility needs to be achieved with CBG and students need to have an opportunity to learn from their mistakes to master a topic, multiple resubmissions are appropriate. Employers are not concerned whether the applicant mastered a topic in Week 2 or after three attempts in Week 10, rather, the employers want to know that a topic was mastered and can be applied. Students should also be given a small freedom to fail and then recover instead of having one low grade drastically affect their overall score. A disadvantage of multiple resubmissions, especially when combined with feedback, is the burden on the instructor to give meaningful and actionable feedback for every submission. Flexibility in CGB is also achieved through flexible deadlines where students can work at their own pace and resubmit until the end of the semester.

Live-coding lab demonstrations

In [8] the authors showed the value of using live-coding as an assessment in programming exams through a hybrid exam. In our work, we apply live-coding to the lab exercise instead of an exam, but the benefits are still the same. In the hybrid-exam methodology [8] students completed a take-home portion of the exam followed by an in-person portion. The in-person portion is used by the instructor to assess the student as well as to determine whether the student engaged in academic misconduct such as working with other students or obtaining code from other sources– if a student cannot answer a question in-person when that question is similar to a take-home question, it is likely that the student did not do their own work on the take-home exam.

This study applied live-coding to the lab exercise portion of the course.

Gamification

Gamification refers to incorporating elements from games into the structure of a course [9]. Gamification can increase student learning, engagement, and enjoyment. In part of this study, we used quests and side-quests, where Lab Exercises were released when Assignments were completed.

3 Methodology

We conducted this study in an undergraduate programming course at Wentworth Institute of Technology. ELEC3150 Object-Oriented Programming for Engineers is an undergraduate programming course that is usually taught in the Fall to our Computer Engineering students by Instructor 1 and in the Summer by Instructor 2 to our Electrical Engineering students. The following semesters' data were used – Instructor 2 in Summer 2019 with 17 students, Instructor 2 in Summer 2020 with 22 students, Instructor 1 in Summer 2021 with 29 students, Instructor 2 in Summer 2021 with 20 students. There were still some pandemic restrictions at Wentworth

Institute of Technology in 2021; Instructor 1 delivered their ELEC3150 Section online while Instructor 2 was in-person. It should be noted that this shows that CBG worked for both the online and in-person versions of the course.

This course has been taught multiple times by both Instructors in the traditional method before this study. This traditional method used both summative (e.g., lab exercises) and formative assessments (e.g., final project). The overall course grade was calculated from weighted course assessment types, for example, lab exercises are worth 30% of the overall course grade, the project 20%, assignments 20%, and quizzes and exams the remaining 30%. Then the individual assessments are graded on a point-score, for example, Assignment 1 is worth 40pts and the midsemester exam is worth 100pts.

This experiment compares several refinements to the original CBG methodology used by another instructor (Instructor 3) in Fall 2020 with 54 students in two Sections.

In CBG, a student's grade is based on a combination of the number of topics completed and the mastery level earned per topic. For example, the following 12 topics were assessed in ELEC3150 in Summer 2021:

- 8 technical topics: flowcharting, coding basics, functions, classes/objects, advanced classes/objects, Standard Template Libraries, data structures, and search/sort algorithms and efficiency.

- 4 non-technical topics: report writing, self-reflection and self-assessment, timeliness, and student effort and success.

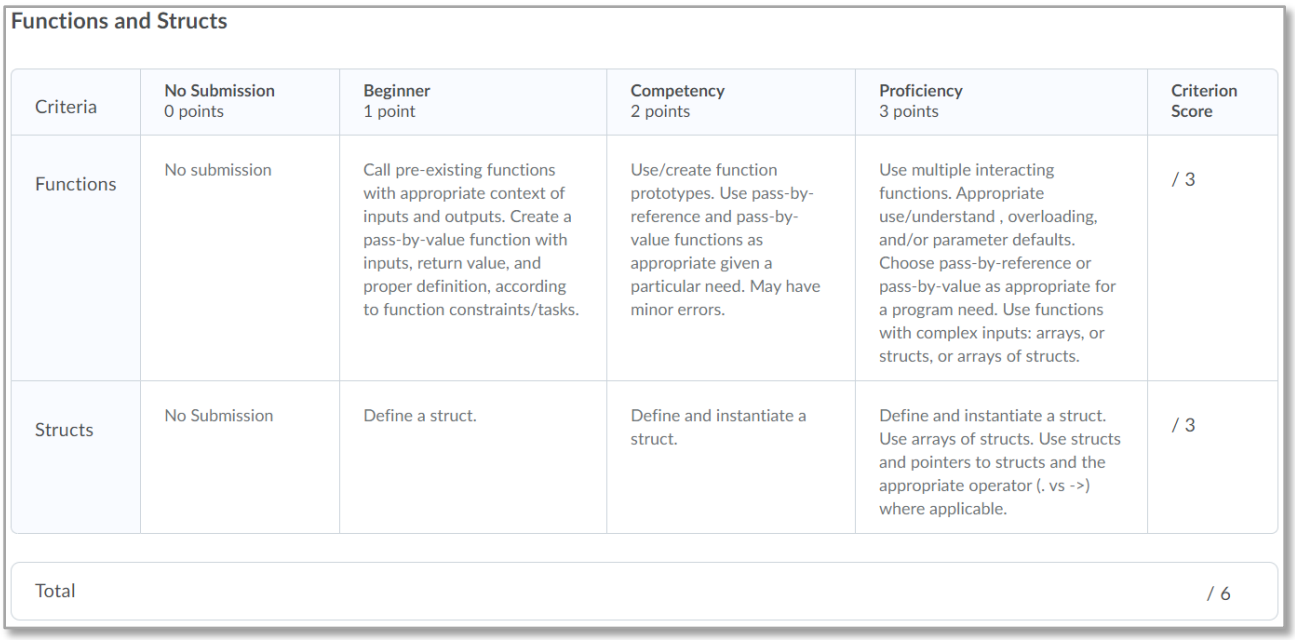
The topics were chosen to match with all instructors in this study. Each topic was graded on a mastery scale of 1 to 3:

- 1 (beginner) – the available evidence does not meet expectations for competency in the topic.

- 2 (competency) – the available evidence meets expectations for competency in the topic.

- 3 (proficiency) – the available evidence exceeds expectations for competency in the topic.

No submission earned a 0. As with most variations of CBG, rubrics were created for all topics. A sample set of rubrics is shown below in Figure 1. In the original CBG methodology, the instructor allowed infinite resubmissions with a flexible deadline and allowed submissions of any assessment until the last week of classes. Instructor 3 also implemented gamification where a quiz grade of > 85% unlocked the assignment, and assignment attempts unlocked the lab exercise for the topic.



Functions and Structs

Criteria	No Submission 0 points	Beginner 1 point	Competency 2 points	Proficiency 3 points	Criterion Score
Functions	No submission	Call pre-existing functions with appropriate context of inputs and outputs. Create a pass-by-value function with inputs, return value, and proper definition, according to function constraints/tasks.	Use/create function prototypes. Use pass-by-reference and pass-by-value functions as appropriate given a particular need. May have minor errors.	Use multiple interacting functions. Appropriate use/understand , overloading, and/or parameter defaults. Choose pass-by-reference or pass-by-value as appropriate for a program need. Use functions with complex inputs: arrays, or structs, or arrays of structs.	/ 3
Structs	No Submission	Define a struct.	Define and instantiate a struct.	Define and instantiate a struct. Use arrays of structs. Use structs and pointers to structs and the appropriate operator (. vs ->) where applicable.	/ 3
Total					/ 6

Figure 1 rubric for one of the [Course 1] lab exercises – this lab exercise used functions and structs each graded out of 3.

The authors made the following observations about Instructor 3's original CBG methodology:

- Most students enjoyed the flexibility of CBG where they could move at their own pace and had a chance to recover from failures.
- Infinite resubmissions increased the instructor's burden because the instructor needed to grade many assignments and had to give feedback in all cases.
- Flexible deadlines could hurt students who have poor time management and increase the instructor burden (grading and feedback).

Table 1 components used in the different trials

	Instructor 1 Refinement 1 CBG	**Instructor 2 Refinement 2 CBG**	**Instructor 3 Original CBG**
Competency-based grading (CBG) including rubrics	✓	✓	✓
Feedback-centered assessment	✓	✓	✓
Infinite resubmissions	✓	×	✓
Flexible deadlines	✓	×	✓
Live-coding demonstration	✓	×	×
Gamification	×	✓	✓

Table 1shows the components used in the different trials. All three instructors used CBG. In Refinement 1, Instructor 1 removed gamification but added live-coding lab demonstrations. In Refinement 2, Instructor 2 replaced infinite resubmissions with one resubmission per assessment, a fixed two-week period to resubmit an assessment after feedback was released to the students

and did not implement live-coding demonstrations. Instructor 2 also only allowed resubmissions to students who made an initial submission before the deadline.

4 Results and Analysis

Refinement 1 – removing gamification and adding live-coding demonstrations

Students are expected to work on their labs individually. The student must do a live demonstration of their working code. During the demonstration, the student must also answer questions about the code and be able to make simple changes such as adding or modifying a function. This live coding demonstration gives an indication of whether the student did their own work and understands the work.

There are many correct solutions to a programming question, using the live-coding discussion, an instructor can determine a student's logic and thought process. This makes grading easier and reduces the instructor's burden. One final advantage is that students receive real-time feedback from the instructor and can immediately ask questions to clarify any feedback given.

Refinement 2 – removing infinite resubmissions and adding fixed deadlines

Figure 2 compares traditional assessment by Instructor 2, CBG methodology for Instructor 1 (Refinement 1), CBG methodology for Instructor 2 (Refinement 2), and the original CBG methodology for Instructor 3. Note that Instructor 3 is the original CBG methodology with infinite resubmission and gamification as shown in Table 1.

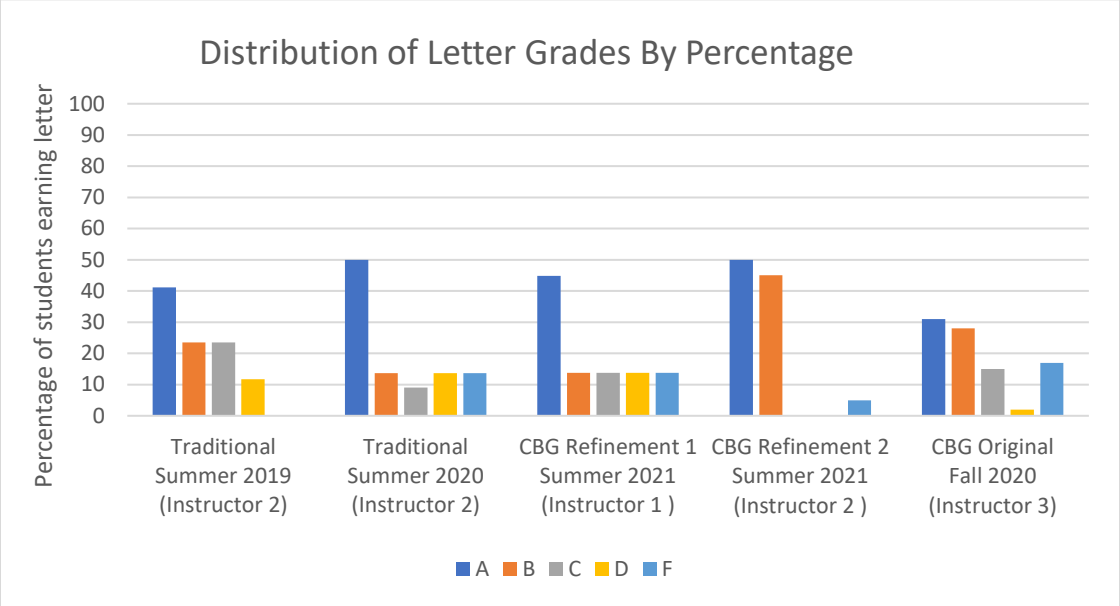


Figure 2 percentage of A, B, C, D, and F grades earned for [Course 1]

Instructor 2 compared their traditional letter grade distribution over two semesters to the letter grade distribution for students using Refinement 2 CBG and observed that the percentage of A's stayed steady, i.e., between 40% and 50%. This means that the students who typically achieve

A's will continue to do so in a CBG structure and CBG does not work as a disadvantage. There was an overall improvement in grades since all other students had B grades (B-, B, B+). The ability to resubmit for a higher grade based on feedback allowed the weaker students to master the topics and do well in the course. The student who received an F did not submit any assessments and did not withdraw from the course before the deadline.

Instructor 2 used the original CBG but removed infinite resubmission and had fixed deadlines. Figure 2 also shows that infinite resubmissions are not necessary for improving grades in a GBG setting since the percentage of A's and pass rate for Instructor 2 is not affected even though the instructor did not allow infinite resubmissions like the other two instructors in the study. Additionally, without infinite resubmissions and flexible deadlines, we achieved the flexibility of CBG while also observing that the instructor burden is reduced compared to the scheme with infinite resubmissions. This was because of less grading for the instructor since the instructor only had to grade one submission and, at most, one resubmission per assessment. The instructor also spent less time giving feedback because they had fewer items to give feedback on. Students still benefited from receiving feedback and being able to resubmit to try for a higher grade and to master the topic.

It was also observed that students learn to pay attention to deadlines (a skill they need for the industry) but also learn that they need to fix their mistakes. With infinite resubmissions and flexible deadlines, students with poor time management skills waited until the last week of classes to submit (or resubmit) all assessments. This is stressful for the students and a time burden for the instructor. The students who waited until the end of the semester were the weaker students who could not complete the assessments quickly, correctly, or in a short period right before the last day of classes, and in most cases barely passed the course. In Refinement two students know when the deadlines are and that they have only one resubmission if they submit an initial attempt. After feedback was posted on the assessment, the instructor let students know that feedback was available and that the resubmission link closed two weeks after the announcement was posted.

Even with one chance for resubmission and fixed deadlines, students still found the course flexible and the CBG methodology helpful as shown in the students' comments from Instructor 2's Course Evaluation for Summer 2021 (emphasis added):

- […] was very helpful throughout the course. If you had a question she would explain things thoroughly. She also kept things realistic, emphasizing practices and concepts that are important to the field, rather than focusing on things that are emphasized at the university level. **Her grading style was optimal for the course, as students are allowed to correct their mistakes rather than punished**.

- Her **teaching style for this course was very effective**. As someone who typically struggles to grasp programming concepts, I was able to **learn and comprehend a lot more** in her class than I thought I would.

- The **flexibility** in this course help make this hard semester a lil [sic] better

An interesting, unwanted, side-effect of infinite resubmissions, was that several students kept resubmitting "blindly" without reflecting on and acting upon the feedback to learn. These resubmissions continued until they finally received enough to pass the course. This left some students unprepared for the follow-up course ELEC3225 Applied Programming Concepts in Summer 2021. Three students who took ELEC3150 in Fall 2021 with the original CBG and infinite resubmissions admitted that they were weak in ELEC3225 because they did not understand ELEC3150's material well since they kept resubmitting until they finally passed without knowing what they were doing. While Refinement 2 removed infinite resubmissions to decrease instructor burden and help students with their time management, it could also result in students taking the pre-requisite material in ELEC3150 more seriously.

5 Conclusion

Competency-based grading (CBG) is an assessment technique that is applicable to skills-based courses such as the programming course used in this study. This paper compared an original CBG methodology that included gamification and infinite resubmissions to two refinements to the CBG methodology. In Refinement 1 gamification was removed and a live-coding demonstration was added. This addition allowed instructors to better understand the student's thought processes and to grade more efficiently. It also discourages students from academic misconduct since students know that they must show that they wrote and understand the code. In Refinement 2 infinite resubmissions were removed, instead, students had one resubmission and fixed deadlines. This refinement still allowed the student flexibility while encouraging time management in students. It also decreased the instructor's burden on grading and releasing feedback.

In future work Refinements 1 and 2 will be combined – keep one resubmission and fixed deadlines and add in live-coding demonstrations for lab exercises. Future work also includes redefining the rubrics to a 4pt scale (rather than 3) to make the translation to the 4.0 GPA an easier calculation.

6 References

[1] ACM & IEEE CC2020 Task Force, "Computing Curricula 2020: Paradigms for Global Computing Education," *Association for Computing Machinery, New York, NY, United States*, April 2021, DOI:https://doi.org/10.1145/3467967

[2] R. Armacost and J. Pet-Armacost, "Using mastery-based grading to facilitate learning*, in 33rd Annual Frontiers in Education, 2003*, *FIE 2003*., volume 1, pages T3A–20. IEEE, 2003.

[3] J. Bekki, O. Dalrymple, and C. Butler, "A mastery-based learning approach for undergraduate engineering programs," *in 2012 Frontiers in Education Conference Proceedings*, pages 1–6. IEEE, 2012.

[4] T. Fernandez, K. Martin, R. Mangum, and C. Bell-Huff, "Whose grade is it anyway?: Transitioning engineering courses to an evidence-based specifications grading system," *in 2020 ASEE Annual Conference & Exposition*, June 2020.

[5] G. Lee-Thomas, A. Kaw, and A. Yalcin, "Using online endless quizzes as graded homework," *in 2011 ASEE Annual Conference & Exposition*, 2011.

[6] V. Viswanathan and M. Calhoun, “Improving student learning experience in an engineering graphics classroom through a rapid feedback and re-submission cycle,” *in 2015 ASEE Annual Conference & Exposition*, June 2015.

[7] B. Paff, “Effect of test retakes on long-term retention,” Master’s thesis, University of Wisconson, 2012.

[8] P. Junsangsri and M. Rawlins, “Combining Take-Home and In-Person Exams to Improve Student Performance and Improve Instructor Grading Efficiency, *The American Society of Engineering Education Northeast Conference, ASEE-2021,* October 2021.

[9] S. Deterding, D. Dixon, R. Khaled, and L. Nacke, “From game design elements to gamefulness: Defining ‘gamification’,” *in Proceedings of the 15th International Academic MindTrek Conference: Envisioning Future Media Environments, MindTrek ’11*, page 9–15, 2011.